

GraphBench Challenge

Lilia KACI

Nada BOUSSETTA

M1 MIAGE APP

12/05/2026

Dépôt Github : https://github.com/nadaBoussetta/Projet_OC

Résultat Heuristique Simple

```
=====
RÉSULTATS FINAUX
=====
Conjectures réfutées : 96/100
Score total          : 867.9
Temps moyen (trouvés): 4.04s
=====
```

Résultat FunSearch

```
=====
RÉSULTATS FINAUX
=====
Conjectures réfutées : 96/100
Score total          : 778.3
Temps moyen (trouvés): 3.11s
=====
```

1. Introduction et objectif

Le système vise à tester et réfuter automatiquement des conjectures en théorie des graphes . Le projet est divisé en deux parties :

1. une **heuristique locale en 3 phases**
2. une **métaheuristique évolutive inspirée de FunSearch**

L'objectif est de trouver rapidement des **contre-exemples de graphes** violant des inégalités entre invariants.

2. Partie 1 : Heuristique simple

L'approche repose sur une recherche incrémentale divisée en trois phases stratégiques, utilisant la bibliothèque **NetworkX** pour manipuler les graphes et un encodage **graph6** pour optimiser les performances.

Pour réduire la charge de calcul, les 25 invariants sont calculés en **lazy evaluation** : seuls ceux nécessaires à la conjecture sont calculés via la fonction `get_needed_invariants`.

Phase 1 — Graphes structurés

Cette phase agit comme un filtre initial ultra-rapide. Au lieu de construire des graphes, le système interroge un catalogue de familles connues :

- Graphes “étoiles” ou “chemins”
- Graphes “Lollipop” et “Barbel”
- Graphes complets et bipartis

Le but est de trouver les cas simples où des violations sont évidentes

Phase 2 — Recherche exhaustive petits graphes

Cette partie couvre l'espace des petits graphes ($n \in [3, 13]$) où les invariants sont calculables presque instantanément.

Nous générons une grande diversité de spécimens (arbres, graphes sans griffe, graphes réguliers) via des tirages aléatoires et des générateurs spécifiques à NetworkX.

Statistiquement, environ **72 % des contre-exemples** se trouvent dans ces petites tailles. Si une violation existe à petite échelle, cette phase la garantit avant de passer à des méthodes plus coûteuses.

Phase 3 — Optimisation locale

Pour les conjectures restantes, On ne cherche plus au hasard, on optimise un candidat :

- **Hill-Climbing & Recuit Simulé** : À partir d'une population de 20 graphes, l'algorithme évalue le "score de violation". Si une mutation améliore ce score, elle est conservée. Le recuit simulé permet de temps en temps d'accepter des mutations dégradantes pour sortir des optimums locaux.
- **Mécanisme** : L'algorithme combine le **crossover** entre les meilleurs spécimens et des **redémarrages adaptatifs** en cas de stagnation. Les mutations ne sont pas aléatoires : elles ciblent la structure (ajout de feuilles, chemins, subdivisions) ou la densité (cliques, triangles)

Réparation : Un module de correction vérifie après chaque modification que le graphe respecte toujours les contraintes de sa classe (Correction de la connexité et maintien des contraintes structurelles)

Fonction de score heuristique :

Le guidage de la recherche s'appuie sur une formule pondérée :

$$F(G) = 25 \cdot \text{violation}(G) + \text{bonus}(\text{ invariants })$$

Cette fonction priorise les graphes violant fortement la conjecture tout en récompensant les structures prometteuses (ex: favoriser le diamètre pour les conjectures de distance).

Exemple :

- domination → favoriser petits γ_t
- matching → favoriser grands couplages
- distances → maximiser diamètre/remoteness

3. Partie 2 : Architecture FunSearch

La seconde partie du système repose sur une architecture inspirée de FunSearch, dont l'objectif est de faire évoluer automatiquement la fonction de score utilisée par l'heuristique. Au lieu d'utiliser une fonction fixe définie manuellement, le système génère et améliore des fonctions de scoring $F\theta(G)$ à l'aide d'un LLM.

Le pipeline se déroule en trois étapes. D'abord, une population initiale de 10 fonctions de score écrites manuellement ("seeds") sert de point de départ. Ensuite, le modèle llama-3.3-70b via l'API Groq génère de nouvelles variantes de fonctions à partir des meilleures fonctions actuelles.

Enfin, une phase de combinaisons offline produit de nouvelles fonctions par recombinaison pondérée des meilleures candidates, sans appel API supplémentaire, ce qui permet d'explorer efficacement l'espace des fonctions tout en limitant le coût et le temps d'exécution.

Résultats FunSearch

FunSearch conserve le même taux de réussite que l'heuristique simple avec 96 conjectures réfutées sur 100, mais améliore les performances globales du système. Le score total passe de **867.9** à **778.3**, soit une amélioration d'environ 10.3%, tandis que le temps moyen d'exécution diminue de 4.04 s à 3.11 s. Après 6 itérations d'évolution et plusieurs phases de recombinaisons offline, la meilleure fonction générée obtient un score d'évaluation environ 26% supérieur à celui de la meilleure fonction seed initiale. Les fonctions produites par FunSearch introduisent notamment une pénalisation des graphes de densité intermédiaire, un bonus favorisant les graphes de taille optimale, ainsi qu'une adaptation dynamique selon les invariants impliqués dans la conjecture.

4. Résultats expérimentaux

Performance globale

Métrique	Simple	FunSearch
Conjectures résolues	96	96
Score total	867.9	778.3
Temps moyen	4.04s	3.11s

Répartition des réussites

La majorité des contre-exemples (72%) sont trouvés rapidement durant les Phases 0 et 1, tandis que 21% nécessitent la Phase 2 et seulement 7% correspondent à des cas difficiles demandant une recherche longue en Phase 2.

Classes de graphes

Les performances varient selon la classe de graphes considérée. Toutes les conjectures sur les arbres sont réfutées (100%), tandis que les graphes claw-free obtiennent un taux de réussite de 98% et les graphes simplement connexes 93%.

Conjectures difficiles

Les conjectures non résolues impliquent principalement les invariants `matching_number` et `total_domination_number`, souvent en interaction avec des invariants de domination ou d'indépendance. Elles ne concernent pas les invariants spectraux. Ces cas semblent particulièrement difficiles car ils combinent des paramètres fortement corrélés par des bornes structurelles connues, limitant l'espace des contre-exemples.4.5

5. Discussion scientifique

Conjectures faciles

Les conjectures les plus faciles à réfuter sont généralement éliminées immédiatement par des graphes très simples comme les étoiles, les cycles ou d'autres petits graphes extrêmes. Dans ces cas, la structure étant très élémentaire, une violation apparaît presque instantanément sans exploration approfondie.

Invariants difficiles

L'invariant le plus difficile à manipuler est la domination totale γ_t , qui apparaît systématiquement dans les cas les plus résistants. Cette difficulté s'explique par ses fortes contraintes structurelles et ses interactions complexes avec d'autres invariants de domination et de couplage.

Mutations efficaces

Les mutations les plus efficaces sont celles qui modifient la structure du graphe de manière contrôlée tout en conservant des propriétés extrêmes. L'ajout de feuilles dans les arbres, la subdivision d'arêtes, l'ajout de chemins ou encore la densification

locale contrôlée se révèlent particulièrement performants pour générer des contre-exemples.

Apport réel de FunSearch

FunSearch apporte une amélioration limitée du système global. Il permet un meilleur guidage sur certaines conjectures et facilite la découverte de nouvelles fonctions de score, mais les gains restent modestes et ne constituent pas une rupture nette par rapport à l'heuristique simple. Dans l'ensemble, les performances restent proches.

Apport scientifique du LLM

Le LLM apporte plusieurs éléments utiles, notamment l'introduction des pondérations adaptatives, des systèmes de paliers dans les fonctions de score et une meilleure prise en compte de la densité optimale des graphes. En revanche, une partie des propositions reste peu pertinente, comme les variations cosmétiques ou les surcomplexifications des fonctions. Globalement, son rôle est surtout celui d'un générateur d'idées à explorer plutôt qu'un moteur d'amélioration radicale des performances.

Conclusion générale

Le système combine une heuristique multi-phase, une recherche locale optimisée et une évolution automatique des fonctions de score via FunSearch, permettant une forte capacité de réfutation (96%), une détection rapide des cas simples et une amélioration globale modérée mais réelle, avec des limites sur les structures complexes liées à la domination et au couplage.